Home Server Remote Access - RustDesk Direct IP

Secure LAN Remote Desktop · Direct IP Access · Windows Firewall · Troubleshooting Log

Oct 13, 2026 · @MrBSykes

Item	Detail
Project Title	Home Server Remote Access: RustDesk Direct IP
Date	October 13, 2026
Analyst	Bryan Sykes
Platform	Windows home server (sykeshomeserver) and a Windows client machine
Remote Access Tool	RustDesk desktop client
Connection Method	Direct IP access over the LAN, TCP 21118, no relay server
Execution Environment	PowerShell and Windows Firewall on the server; RustDesk on both machines
GitHub	github.com/MrBSykes
Coverage	Business problem, architecture, implementation walkthrough, troubleshooting log, business alignment

1. The Business Problem

Remote support is only as reliable as the path between the technician and the machine. Help desk and IT support staff constantly need to reach systems they are not sitting at: servers, user workstations, lab machines. Remote desktop tools make that possible, but when the connection depends on infrastructure you do not control, access can disappear without warning.

That is what happened here. Connecting to the home server by RustDesk ID through the free public relay failed with a "connection is not allowed" error and a prompt to log in. The public server now requires a login because of scam and botnet abuse, and its operators describe it as a demo and testing service, not something to use for production or sensitive work. Three problems follow from relying on it:

- **Third-party dependency:** access can be restricted or withdrawn at any time, as this error showed.
- **Exposure:** sessions through a shared public relay are a poor fit for a home server that holds lab data.
- **Reliability:** a self-hosted relay (hbbs/hbbr in Docker) was tried first and never connected, so a simpler path was needed.

Direct IP access removes the relay entirely. The client connects straight to the server's LAN address, with no account, no third party and nothing leaving the network.

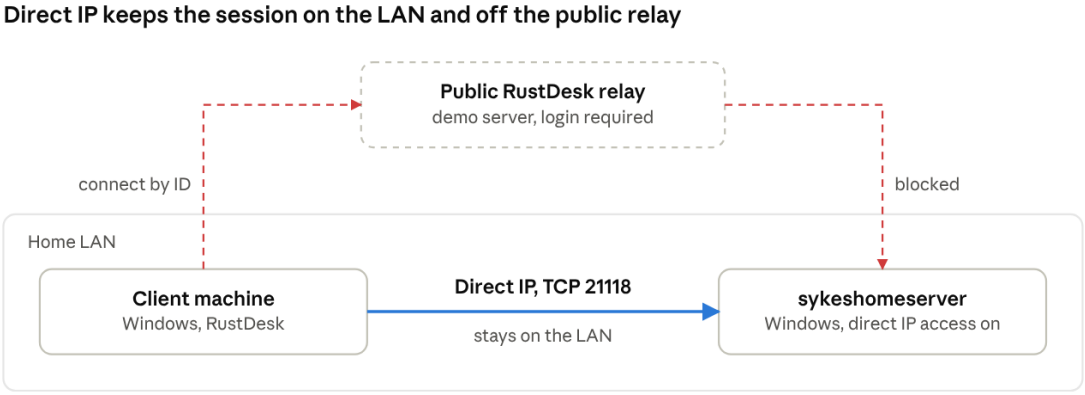
1.1 Real-World Applicability

Scenario	Without a controlled access path	With direct IP access
Vendor restricts a free service	Support sessions stop until a workaround is found	Connection path is unaffected
Support session to an internal server	Traffic crosses a third-party relay	Traffic stays on the LAN
Reviewing who can reach a machine	Shared infrastructure, unclear exposure	One firewall-scoped port plus a password
"I can't connect" ticket	Trial and error	Layered checks: listener, firewall, client path
Onboarding a new technician	Tribal knowledge	A written runbook

2. Architecture

The final design is one direct TCP connection between two machines on the same network, with the public relay out of the path.

2.1 Connection Topology



connection paths · 1 working, 1 blocked

The dashed red route is the original connect-by-ID attempt through the public relay, which now requires a login. The solid route is the direct IP session that works.

2.2 Security Design Principles

- **Least exposure:** one inbound port (TCP 21118), allowed in Windows Firewall, with no router port forwarding, so it is reachable from the LAN only.
- **No third party in the path:** no relay and no account sits between the two machines.
- **Authenticated sessions:** a password is required to connect, and a permanent password replaces the one-time one.
- **Public relay kept out of sensitive work:** this follows the operators' own guidance that the public server is for demos only.
- **Outside access deferred:** reaching the server from outside the home waits for a mesh VPN such as Tailscale, not opened ports.

3. Implementation Walkthrough

The working setup is four steps on the server and two on the client. The one rule that matters: connect by **IP address, never by RustDesk ID**, because an ID connection goes through the public server and hits the login block again.

3.1 Server (sykeshomeserver)

1. Open RustDesk, go to Settings, then Security, and click **Unlock security settings**.
2. Turn on **Enable direct IP access**.
3. Allow inbound TCP 21118 in Windows Firewall, from an admin PowerShell:

```
New-NetFirewallRule -DisplayName "RustDesk Direct" -Direction Inbound -Protocol TCP -LocalPort 21118 -Action Allow -Profile Any
```

4. Set a permanent password under Settings, then Security, so access does not depend on the one-time password.

3.2 Client

1. In the "Control Remote Desktop" box, enter the server's LAN IP address (for example `192.168.1.x`).
2. Click **Connect** and enter the permanent password.

3.3 Verification Commands

Run these when a connection fails, in order. Each one isolates a single layer.

```
# On the server: is anything listening on the direct IP port?
netstat -an | findstr 21118

# On the client: can it reach that port?
Test-NetConnection <SERVER_LAN_IP> -Port 21118
```

No listener means direct IP access is not actually enabled. A listener but a failed `Test-NetConnection` points to the firewall, router client isolation, or a different subnet.

3.4 Display Tuning

The first session showed black bars and a shrunken desktop. These session settings address it:

- Switch to fullscreen from the session toolbar.
- Set **View Style** to *Scale adaptive* (fits the window) or *Original* (1:1 pixels, scrolls).
- Set **Image Quality** to *Best*, since a LAN has plenty of bandwidth.
- If text is still small, lower the display resolution on the server.

4. Components and Status

Direct IP access is the one path that works; both relay-based options were set aside.

Component	Detail	Status
Public RustDesk relay	Connect by ID through the demo server	Blocked, login required
Self-hosted hbbs/hbbr in Docker	Own relay on the home server	Abandoned after repeated connection failures
Direct IP access on sykeshomeserver	RustDesk setting, TCP 21118 with a firewall allow rule	Working
Client session display settings	Fullscreen, view style, image quality	Tuned
Permanent password	Replaces the one-time password	Recommended
DHCP reservation for the server	Keeps the saved LAN address valid	Planned

5. Troubleshooting Log

Four issues came up. Each is recorded with its root cause (or an honest note where it was not isolated) and the lesson a support team would carry forward.

Issue 1: Public Server Login Required

Field	Detail
Symptom	Connection error: the connection is not allowed, and the user must log in before connecting
Root Cause	The RustDesk public server now requires a login because of scam and botnet abuse, and is meant for demos only. The client was still pointed at it, as the status bar prompt to set up your own server showed.
Resolution	Stopped relying on the public relay and moved to direct IP access

Field	Detail
Business Lesson	Free third-party services change their terms without notice. Do not build a support workflow on a service that is explicitly demo-only.

Issue 2: Self-Hosted Relay in Docker Would Not Connect

Field	Detail
Symptom	Repeated failed connections using hbbs and hbbr containers on the home server
Root Cause	Not isolated. Common causes for this setup are UDP 21116 not published, a missing or wrong public key on a client, and no relay address set on hbbs.
Resolution	Dropped the Docker route and used direct IP access instead. A native Windows hbbs/hbbr install is a follow-up.
Business Lesson	Choose the simplest architecture that meets the requirement, time-box the complex one, and record honestly what was not diagnosed.

Issue 3: Direct IP Connection Failed at First

Field	Detail
Symptom	Connecting by IP address failed on early attempts
Root Cause	Not recorded at the time. Candidates checked: direct IP access not enabled (no listener on 21118), a firewall blocking 21118, or an ID typed where an IP belongs.
Resolution	Worked through the layered checks (netstat, then Test-NetConnection), after which the session connected
Business Lesson	Test one layer at a time: listener, firewall, then client path. Random setting changes hide the cause.

Issue 4: Compressed Remote Display

Field	Detail

Field	Detail
Symptom	The remote desktop appeared small with black bars on both sides
Root Cause	The server's screen shape and resolution differed from the client window, and the default view style scaled the image down to fit
Resolution	Went fullscreen, changed the view style, and set image quality to Best
Business Lesson	A connection that works is not yet a usable support session. Tune the display before handing it to a technician.

6. Business and Career Alignment

6.1 Help Desk and Federal IT Relevance

Remote desktop sessions are daily work on a help desk or service desk, and the technician who understands what sits underneath one (ports, firewalls, relay versus direct paths) can fix a "can't connect" ticket instead of escalating it.

- **Service desk and Tier 1 support:** the troubleshooting sequence here (listener, firewall, client path) is the same sequence used on real connectivity tickets.
- **Federal contracting:** NIST 800-53 control AC-17 (Remote Access) expects remote access methods to be documented, authorized and restricted. A single named port, a password and a written runbook support that.
- **Least exposure:** the final design keeps traffic on the LAN with no router port forwarding and no third-party relay.

6.2 Skills Demonstrated

Skill	Evidence in This Project
Remote support tooling	Configured and tuned RustDesk sessions between two Windows machines
Windows networking and firewall	Created an inbound rule for TCP 21118 in PowerShell and checked listeners with netstat
Structured troubleshooting	Isolated the listener, firewall and client path with netstat and Test-NetConnection
Reading error messages	Interpreted the public server's login block and chose a different design
Documentation	This write-up, including a troubleshooting log

Skill	Evidence in This Project
	with business context

6.3 Certification Alignment

Certification	Relevance to This Project
CompTIA Security+ (earned)	Secure remote access, least exposure, avoiding a public relay for sensitive work, port and firewall rules
AZ-104 (next)	Azure Bastion, planned in the Azure project, solves the same secure-remote-access problem in the cloud

7. Next Steps

Action	Purpose
Reserve a fixed IP for the server in the router	Keep the saved LAN address valid after reboots
Add Tailscale to both machines	Reach the server from outside the home network without port forwarding
Rebuild the self-hosted relay natively on Windows	Finish the path that failed in Docker and find its real root cause
Run hbbs and hbbr as Windows services	Make the relay survive reboots if it is rebuilt
Publish this write-up with the README on GitHub	Add it to the home lab portfolio

8. Conclusion

This project treated a blocked third-party service as a design input, not a dead end. Direct IP access over the LAN gives remote desktop access to the home server with no account, no relay and no exposed internet port.

The problems along the way were ordinary support work: an error message that explained its own cause, a complex path abandoned for a simpler one, and a connection fault narrowed down one layer at a time. Diagnosing and documenting them is the job.